

Flow Control at the Data Link Layer using the Sliding Window Protocol

AIM

To write a program to implement **Flow Control at the Data Link Layer using the Sliding Window Protocol** and simulate the flow of frames from one node to another. The program also verifies acknowledgements and retransmits frames when an error or NACK is received.

SENDER

The sender performs the following operations:

1. Accepts the **window size** from the user.
2. Accepts a **text message** from the user.
3. Divides the message into frames, considering **one character per frame**.
4. Each frame contains:
 - Frame Number
 - DATA
5. Writes the frames to `Sender_Buffer.txt`.
6. Sends the frames according to the specified window size.
7. Waits for the receiver's acknowledgement.
8. Reads `Receiver_Buffer.txt` and checks the ACK/NACK number.
9. If a correct ACK is received, the sender transmits the next set of frames.
10. If a NACK is received, the sender retransmits the required frames.

RECEIVER

The receiver performs the following operations:

1. Reads the frames from `Sender_Buffer.txt`.
2. Checks the frame number of each received frame.
3. Compares the received frame number with the expected frame number.
4. If the frame number is correct, it generates an **ACK**.
5. If the frame number is incorrect, it generates a **NACK**.
6. The ACK/NACK is written into `Receiver_Buffer.txt`.
7. The sender reads this file and performs the required transmission or retransmission

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#define MAX 100
```

```
struct Frame
```

```
{
```

```
    int frame_no;
```

```
    char data;
```

```
};
```

```
/* Write frames to Sender_Buffer */
```

```
void writeSenderBuffer(struct Frame frames[], int start, int end)
```

```
{
```

```
    FILE *fp = fopen("Sender_Buffer.txt", "w");
```

```
    if (fp == NULL)
```

```
    {
```

```
        printf("Error opening Sender_Buffer.txt\n");
```

```
        exit(1);
```

```
    }
```

```
    fprintf(fp, "Frame_No DATA\n");
```

```
    for (int i = start; i < end; i++)
```

```
    {
```

```
        fprintf(fp, "%d %c\n",
```

```

        frames[i].frame_no,
        frames[i].data);
    }

    fclose(fp);
}

/* Read ACK/NACK from Receiver_Buffer */
int readReceiverBuffer()
{
    FILE *fp;
    char field[20];
    int ack;

    fp = fopen("Receiver_Buffer.txt", "r");

    if (fp == NULL)
    {
        printf("Receiver_Buffer.txt not found!\n");
        return -1;
    }

    fscanf(fp, "%s %d", field, &ack);

    fclose(fp);
}

```

```
    return ack;
}

/* Receiver */

void receiver()
{
    FILE *senderFile;

    FILE *receiverFile;

    int frameNo;

    char data;

    int expected = 0;

    senderFile = fopen("Sender_Buffer.txt", "r");

    if (senderFile == NULL)
    {
        printf("Sender_Buffer.txt not found!\n");

        return;
    }

    receiverFile = fopen("Receiver_Buffer.txt", "w");

    if (receiverFile == NULL)
```

```

{

    printf("Unable to create Receiver_Buffer.txt\n");

    fclose(senderFile);

    return;

}


/* Skip header */

fscanf(senderFile, "%*s %*s");


printf("\n===== RECEIVER =====\n");


while (fscanf(senderFile, "%d %c",

               &frameNo, &data) == 2)

{

    printf("Received Frame No: %d, DATA: %c\n",

           frameNo, data);


    if (frameNo == expected)

    {

        printf("Frame %d received correctly.\n",

               frameNo);


        fprintf(receiverFile,

                "ACK %d\n",

                expected + 1);

```

```

        expected++;
    }
    else
    {
        printf("ERROR: Expected Frame %d "
               "but received Frame %d\n",
               expected, frameNo);

        fprintf(receiverFile,
                "NACK %d\n",
                expected);

        fclose(senderFile);
        fclose(receiverFile);
        return;
    }
}

fclose(senderFile);
fclose(receiverFile);

printf("Receiver Buffer updated successfully.\n");
}

```

```
/* Sender */  
  
void sender()  
{  
  
    char message[MAX];  
  
    int windowSize;  
  
    int length;  
  
    int base = 0;  
  
    int nextFrame;  
  
    int ack;  
  
  
    struct Frame frames[MAX];  
  
  
    printf("Enter Window Size: ");  
  
    scanf("%d", &windowSize);  
  
  
    getchar();  
  
  
    printf("Enter Text Message: ");  
  
    fgets(message, MAX, stdin);  
  
  
    message[strcspn(message, "\n")] = '\0';  
  
  
    length = strlen(message);  
  
  
    /* Create one frame per character */
```

```

for (int i = 0; i < length; i++)
{
    frames[i].frame_no = i;

    frames[i].data = message[i];
}

printf("\n===== SENDER =====\n");

while (base < length)
{
    nextFrame = base + windowSize;

    if (nextFrame > length)
        nextFrame = length;

    printf("\nSending Frames %d to %d\n",
        base, nextFrame - 1);

    /* Write current window */

    writeSenderBuffer(frames, base, nextFrame);

    printf("\nSender_Buffer contents:\n");

    for (int i = base; i < nextFrame; i++)
    {

```

```
    printf("Frame No: %d\tDATA: %c\n",  
           frames[i].frame_no,  
           frames[i].data);  
}
```

```
printf("\nWaiting for Receiver...\n");
```

```
/* Delay */
```

```
sleep(2);
```

```
receiver();
```

```
sleep(1);
```

```
/* Read ACK/NACK */
```

```
ack = readReceiverBuffer();
```

```
if (ack == -1)
```

```
    return;
```

```
if (ack > base && ack <= nextFrame)
```

```
{
```

```
    printf("\nACK %d received.\n", ack);
```

```
    base = ack;
```

```

    }

    else

    {

        printf("\nNACK %d received.\n", ack);


        printf("Resending frames from Frame %d...\n",
            ack);


        base = ack;


        if (base < 0 || base >= length)

            base = 0;

    }

}


printf("\n=====\n");
printf("All frames transmitted successfully!\n");
printf("Sliding Window transmission completed.\n");
printf("=====\n");
}


int main()

{

    printf("=====\n");

    printf("  SLIDING WINDOW PROTOCOL SIMULATION\n");

```

```

printf("=====\\n");

sender();

return 0;

}

```

INPUT

```

Enter Window Size: 3
Enter Text Message: HELLO

```

The message is divided into individual frames:

```

Frame 0 → H
Frame 1 → E
Frame 2 → L
Frame 3 → L
Frame 4 → O

```

OUTPUT

```

===== SENDER =====

```

```

Sending Frames 0 to 2

```

```

Sender_Buffer contents:

```

```

Frame No: 0    DATA: H
Frame No: 1    DATA: E
Frame No: 2    DATA: L

```

```

Waiting for Receiver...

```

```

===== RECEIVER =====

```

```

Received Frame No: 0, DATA: H
Frame 0 received correctly.

```

```

Received Frame No: 1, DATA: E
Frame 1 received correctly.

```

```

Received Frame No: 2, DATA: L
Frame 2 received correctly.

```

```

Receiver Buffer updated successfully.

```

```

ACK 3 received.

```

```

Sending Frames 3 to 4

```

Sender_Buffer contents:

Frame No: 3 DATA: L
Frame No: 4 DATA: O

Waiting for Receiver...

===== RECEIVER =====

Received Frame No: 3, DATA: L
Frame 3 received correctly.

Received Frame No: 4, DATA: O
Frame 4 received correctly.

ACK 5 received.

=====
All frames transmitted successfully!
Sliding Window transmission completed.
=====

RESULT

Thus, the **Sliding Window Protocol** was successfully implemented to provide flow control at the data link layer. Frames were transmitted according to the specified window size, acknowledgements were verified, and erroneous frames were retransmitted using NACK information.